Multimedia Computing

By

Minhaz Uddin Ahmed, PhD

Department of Computer Engineering

Inha University in Tashkent.

Email: minhaz.ahmed@gmail.com

Content

- Chapter - 8 (Gonzalez books)
- Image compression
- Discrete Cosine Transform (DCT)
- Coding Redundancy
- Interpixel Redundancy
- Psychovisual Redundancy
- Lossy and lossless example

Objective

- Reduce the number of bytes required to represent a digital image
  - Redundant data reduction
  - Remove patterns
  - Uncorrelated data confirms redundant data elimination

Enabling Technology

- Compressions is used in
  - FAX
  - Teleconference
  - REMOTE DEMO
  - etc

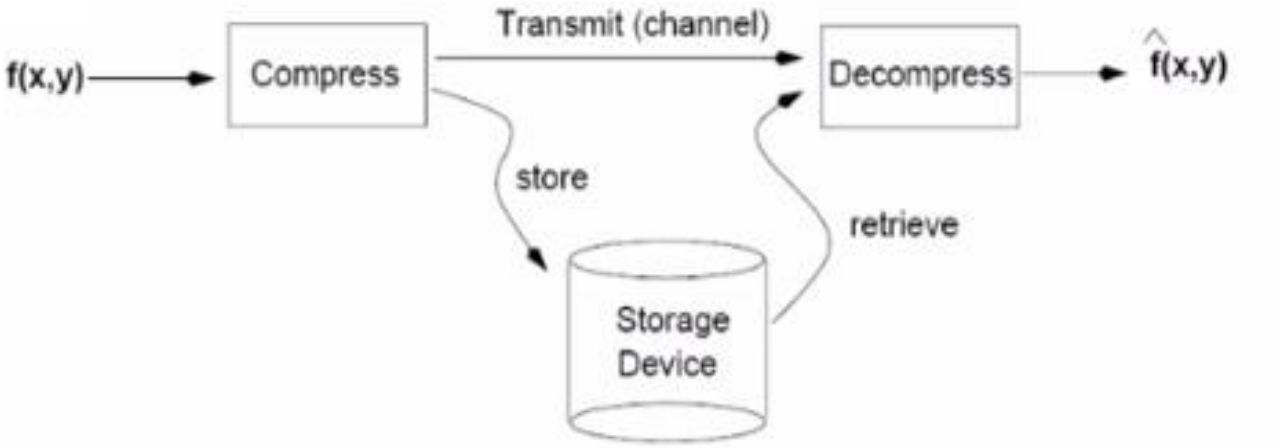
Review

- What and how to exploit data redundancy
- Model based approach to compression
- Information theory principles

- Types of compression
  - Lossless, lossy

Image compression

Image compression is the process of reducing the amount of data required to represent an image.



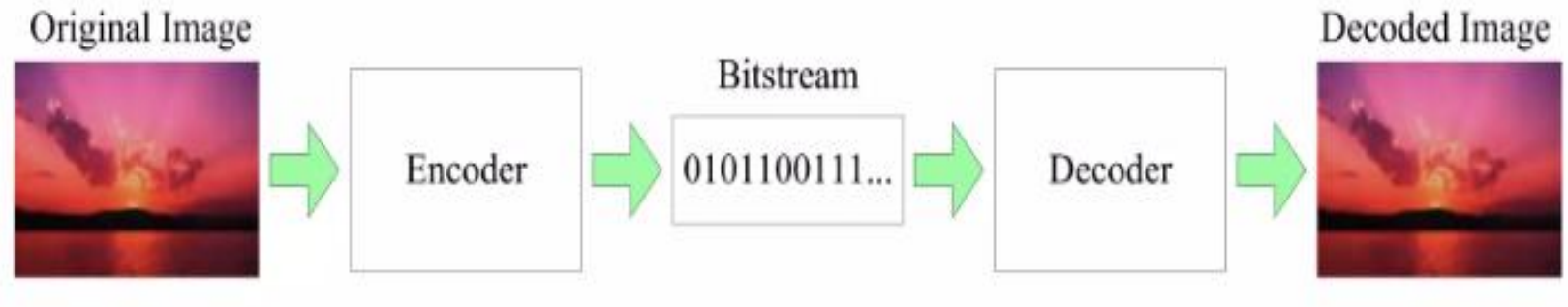
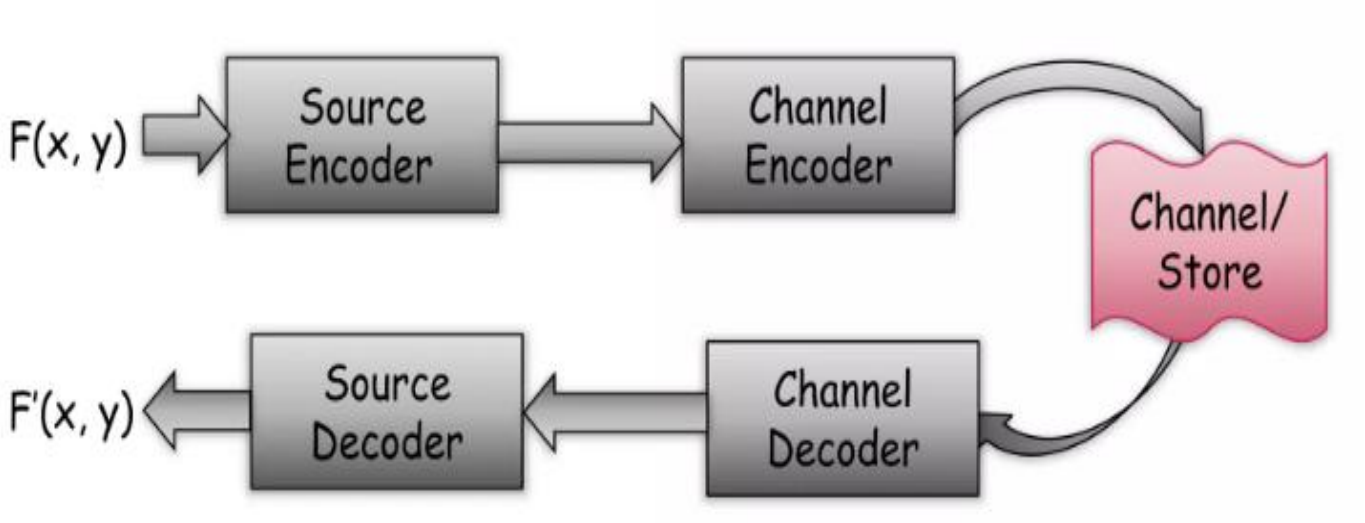


Image compression general models



Some Image compression standard

- JPEG-based on DCT (discrete cosine transform)
- JPEG 2000 based on DWT (discrete wavelet transform)
- GIF-graphics Interchange Format etc.

Discrete cosine transform

- The **Discrete Cosine Transform (DCT)** is a widely used technique in image and signal processing, especially for compression. It transforms a signal or image from the **spatial domain** (e.g., pixel intensities) to the **frequency domain**. The DCT is primarily used in image compression techniques such as **JPEG** because it has the property of concentrating most of the image's information in a small number of frequency components.
- DCT transforms a set of data points (e.g., pixels in an image) into a sum of cosine functions oscillating at different frequencies.
- In image compression, the DCT is used to decompose an image into its frequency components, where the less important (higher frequency) components can be discarded, significantly reducing the size of the image while maintaining a similar visual appearance.

2D Discrete Cosine Transform (DCT):

- For images, the **2D DCT** is used to transform blocks of the image (typically 8x8 pixel blocks). The DCT transforms the pixel intensities into a set of frequency coefficients.

The general form for the 2D DCT of an image block $f(x, y)$ (for example, an 8x8 block) is:

$$F(u,v) = \frac{1}{4} \sum_{x=0}^{N-1} \sum_{y=0}^{N-1} f(x,y) \cdot \alpha(u) \cdot \alpha(v) \cdot \cos\left(\frac{(2x+1)u\pi}{2N}\right) \cdot \cos\left(\frac{(2y+1)v\pi}{2N}\right)$$

Where:

- $f(x, y)$ is the pixel value at location (x, y) in the image block.
- $F(u, v)$ is the DCT coefficient for frequency (u, v).
- N is the block size (usually 8 for JPEG).
- $\alpha(u)$ and $\alpha(v)$ are normalizing factors:

$$\alpha(u) = \begin{cases} \frac{1}{\sqrt{2}} & \text{if } u = 0 \\ 1 & \text{if } u \neq 0 \end{cases}$$

2D Discrete Cosine Transform (DCT):

- Steps in Image Compression with DCT:

- 1.Block the image: Split the image into non-overlapping 8x8 blocks.
- 2. Apply 2D DCT: For each block, compute the DCT to transform the spatial domain pixel values into frequency domain coefficients.
- 3.Quantization: The DCT coefficients are quantized to reduce the amount of data. This step introduces compression by discarding less important high-frequency components.
- 4.Zigzag scan: The quantized coefficients are ordered in a zigzag pattern to prioritize low-frequency coefficients.
- 5.Entropy encoding: Techniques like Huffman coding are applied to further compress the data.

DCT Example

Color image

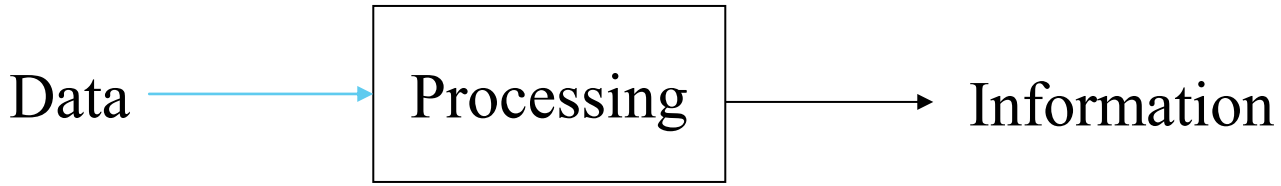
Original Image

DCT Transformed Image

DCT Example

```
 % Read the image
I = imread('12.jpg');
I_gray = rgb2gray(I); % Convert to grayscale if it's a color image
% Perform DCT on the 8x8 blocks
block_size = 8;
dct_transform = @(block_struct) dct2(block_struct.data);
DCT_image = blockproc(I_gray, [block_size block_size], dct_transform);  %%computes the 2D DCT in MATLAB.
% Display original and DCT-transformed images
subplot(1, 2, 1);
imshow(I_gray);
title('Original Image');
 subplot(1, 2, 2);
imshow(log(abs(DCT_image)), []);
title('DCT Transformed Image');
```

Information recovery



- We want to recover the information, with reduced data volumes.
- Reduce data redundancy.
- How to measure the data redundancy.

  - Image compression is an irreversible process.
  - Some Transform used in Image Compression
    - DCT-Discrete Cosine Transform
    - DWT-Discrete wavelet Transform etc.

Relative Data Redundancy

- Assume that we have two data sets D_1 and D_2.
  - Both on processing yield the same information.
  - Let n_1 and n_2 be the info – carrying units of the respective data sets.
  - Relative data redundancy is defined on comparing the relative dataset sizes

$R_D = 1 - 1/C_R$

where C_R is the compression ratio

$C_R = n_1 / n_2$

Relative Data Redundancy

Original Image

JPEG Compressed 90% Quality

JPEG Compressed 50% Quality

JPEG Compressed 10% Quality

Code lossy

```
>> img = imread('peppers.png');
>> imshow(img), title('Original Image');
>> % Step 2: Save the image with different JPEG quality levels
imwrite(img, 'compressed_image_90.jpg', 'jpg', 'Quality', 90); % 90% quality
imwrite(img, 'compressed_image_50.jpg', 'jpg', 'Quality', 50); % 50% quality
imwrite(img, 'compressed_image_10.jpg', 'jpg', 'Quality', 10); % 10% quality

>> % Step 3: Read and display the compressed images
img_90 = imread('compressed_image_90.jpg');
img_50 = imread('compressed_image_50.jpg');
img_10 = imread('compressed_image_10.jpg');
>> figure;
subplot(2, 2, 1), imshow(img), title('Original Image');
subplot(2, 2, 2), imshow(img_90), title('JPEG Compressed 90% Quality');
subplot(2, 2, 3), imshow(img_50), title('JPEG Compressed 50% Quality');
subplot(2, 2, 4), imshow(img_10), title('JPEG Compressed 10% Quality');
>>
```

Lower quality values result in more compression but also greater loss of image fidelity.

Loss less

Original Image

PNG Compressed (Lossless)

PNG uses lossless compression, meaning the compressed image will be exactly the same as the original in terms of pixel values, but with a smaller file size.

Code lossless

```
>> img = imread('peppers.png');
>> imwrite(img, 'compressed_image_lossless.png', 'png');
>> img_png = imread('compressed_image_lossless.png');
>> figure;
subplot(1, 2, 1), imshow(img), title('Original Image');
subplot(1, 2, 2), imshow(img_png), title('PNG Compressed (Lossless)');
```

File size of compressed image

```
% Check the file sizes of the compressed images
original_info = dir('peppers.png');
jpeg_90_info = dir('compressed_image_90.jpg');
jpeg_50_info = dir('compressed_image_50.jpg');
jpeg_10_info = dir('compressed_image_10.jpg');
png_info = dir('compressed_image_lossless.png');


fprintf('Original Image Size: %d bytes\n', original_info.bytes);
fprintf('JPEG 90%% Quality Size: %d bytes\n', jpeg_90_info.bytes);
fprintf('JPEG 50%% Quality Size: %d bytes\n', jpeg_50_info.bytes);
fprintf('JPEG 10%% Quality Size: %d bytes\n', jpeg_10_info.bytes);
fprintf('PNG Lossless Size: %d bytes\n', png_info.bytes);
```

Original Image Size: JPEG 90% Quality Size: 41327 bytes JPEG
50% Quality Size: 15671 bytes JPEG
10% Quality Size: 6619 bytes PNG Lossless Size: 287589 bytes

Examples

$R_D = 1 - 1/C_R$

$C_R = n_1 / n_2$

- D_1 is the original and D_2 is compressed.
- When $C_R = 1$, i.e. $n_1 = n_2$ then $R_D=0$; no data redundancy relative to D_1 .
- When $C_R = 10$, i.e. $n_1 = 10\ n_2$ then $R_D=0.9$; implies that 90% of the data in D_1 is redundant.
- What does it mean if $n_1 << n_2$?

Types of data redundancy

- Coding Redundancy– Occurs when the data used to represent the image is not utilized in an optimal manner
- Interpixel Redundancy – Occurs because adjacent pixels tend to be highly correlated, in most images the brightness levels do not change rapidly, but change gradually.
- Psychovisual Redundancy- Some information is more important to the human visual system than other type of information.

Coding Redundancy

- How to assign codes to alphabet
- In digital image processing
  - Code = gray level value or color value
  - Alphabet is used conceptually
- General approach
  - Find the more frequently used alphabet
  - Use fewer bits to represent the more frequently used alphabet, and use more bits for the less frequently used alphabet

Coding Redundancy 2

- Focus on gray value images
- Histogram shows the frequency of occurrence of a particular gray level
- Normalize the histogram and convert to a pdf representation
- let r_k be the random variable

  $p_r(r_k) = n_k/n$; k = 0, 1,2 …., L-1, where L is the number of gray level values

  $l(r_k)$ = number of bits to represent r_k

  $L_{avg} = \sum_{k=0 \text{ to } L-1} l(r_k)\, p_r(r_k)$ = average number of bits to encode one pixel. For M x N image, bits required is MN L_{avg}

  For an image using an 8 bit code, $l(r_k) = 8$, $L_{avg} = 8$.

  Fixed length codes.

Fixed vs Variable Length Codes

r_k	$p_r(r_k)$	Code 1	$l_1(r_k)$	Code 2	$l_2(r_k)$
$r_0 = 0$	0.19	000	3	11	2
$r_1 = 1/7$	0.25	001	3	01	2
$r_2 = 2/7$	0.21	010	3	10	2
$r_3 = 3/7$	0.16	011	3	001	3
$r_4 = 4/7$	0.08	100	3	0001	4
$r_5 = 5/7$	0.06	101	3	00001	5
$r_6 = 6/7$	0.03	110	3	000001	6
$r_7 = 1$	0.02	111	3	000000	6

TABLE 8.1
Example of variable-length coding.

Lavg = 2.7

$C_R = 3/2.7 = 1.11$

$R_D = 1 - 1/1.11 = 0.099$

$$CR = \frac{\text{Original Size}}{\text{Compressed Size}}$$

$$RD = 1 - \frac{\text{Compressed Size}}{\text{Original Size}}$$

Code assignment view

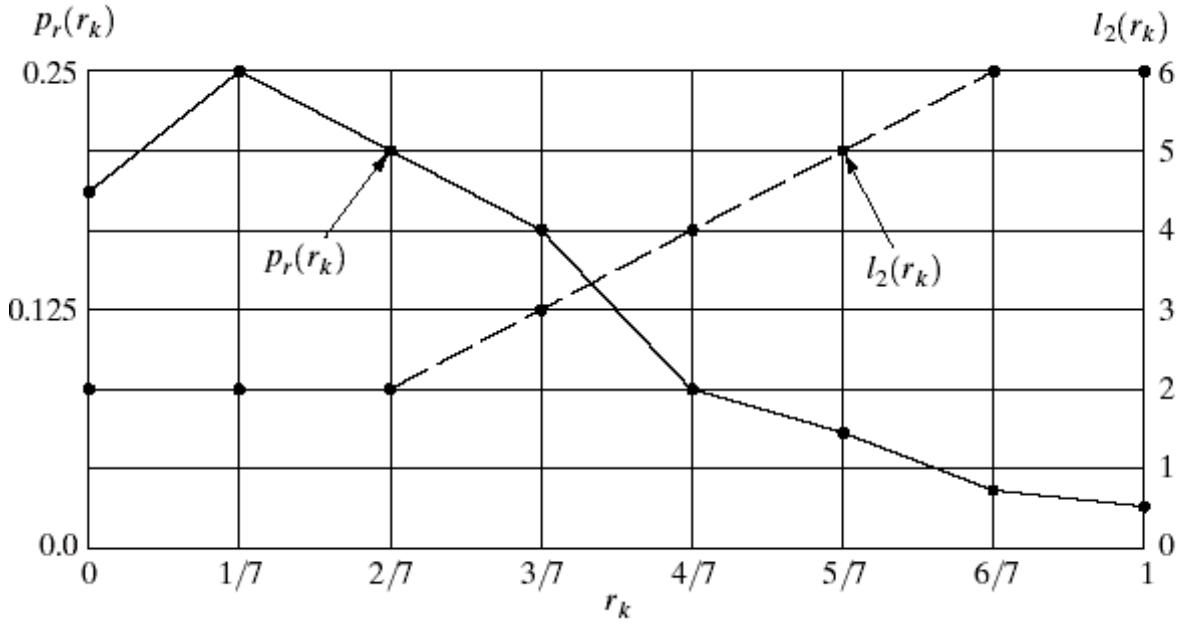
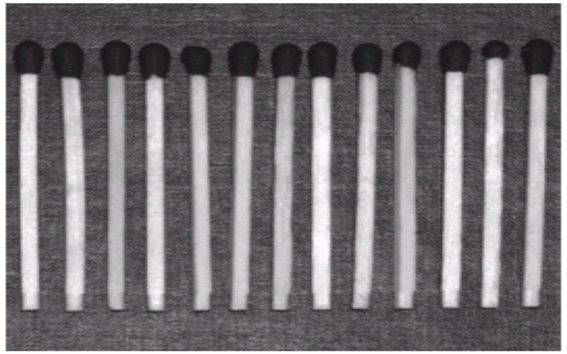
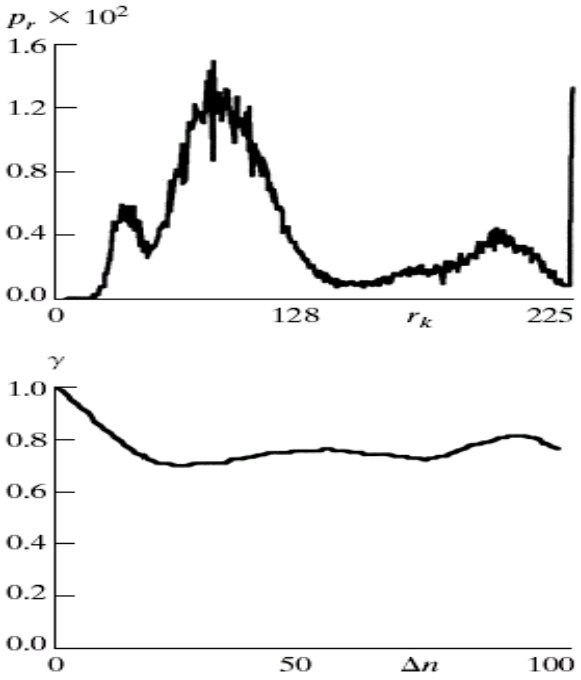
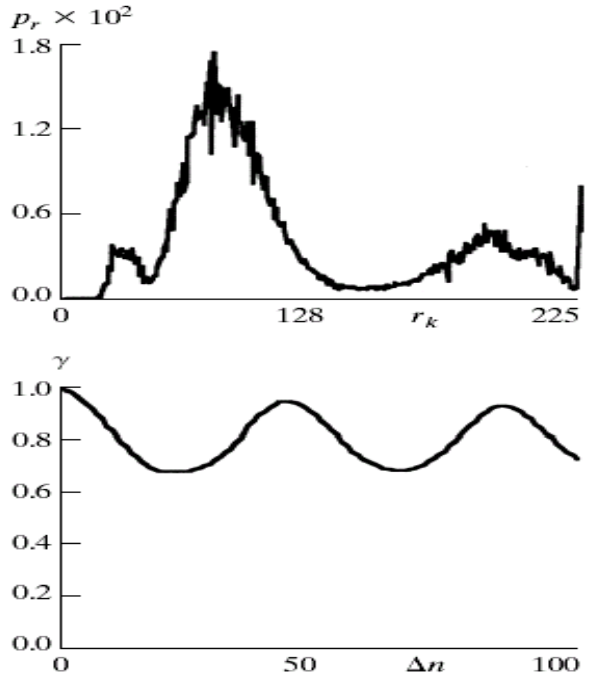


FIGURE 8.1 Graphic representation of the fundamental basis of data compression through variable-length coding.

Interpixel Redundancy







a b
c d
e f

FIGURE 8.2 Two images and their gray-level histograms and normalized autocorrelation coefficients along one line.

Interpixel Redundancy

Inter-pixel Spatial redundancy:

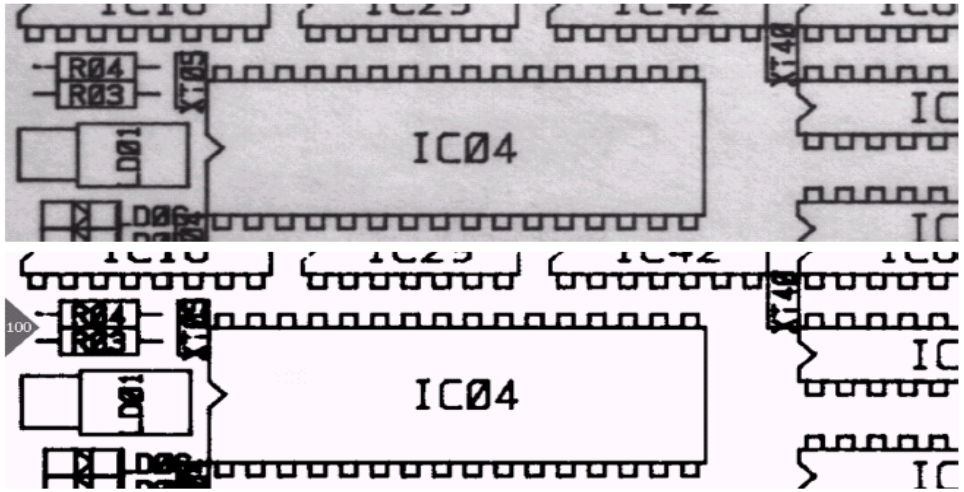
- Inter-pixel redundancy is due to the correlation between the neighboring pixels in an image.
- The value of any given pixel can be predicted from the value of its neighbors(Highly correlated)
- The information carried by individual pixel is relatively small.
- To reduce inter-pixel redundancy the difference between adjacent pixels can be used to represent an image.

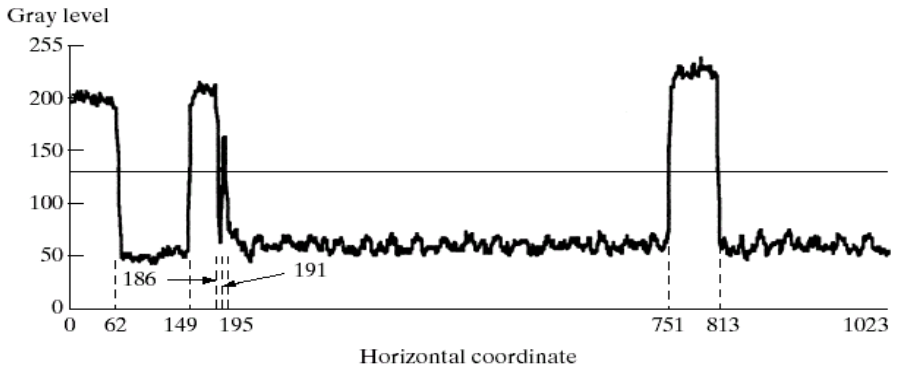
Inter-pixel Temporal Redundancy

- Inter-pixel temporal redundancy is the statistical correlation between pixels from successive frames in video sequence.
- Temporal redundancy is also called inter-frame redundancy.
- Removing a large amount of redundancy leads to efficient video compression.

Algorithm: Run Length Coding

Run Length Coding





Line 100: (1, 63) (0, 87) (1, 37) (0, 5) (1, 4) (0, 556) (1, 62) (0, 210)

a
b
c
d

FIGURE 8.3
Illustration of run-length coding: (a) original image. (b) Binary image with line 100 marked. (c) Line profile and binarization threshold. (d) Run-length code.

CR=1024*343/12166*11
= 2.63
RD = 1-1/2.63 = 0.62

Run Length Coding

- Run Length Encoding (RLE) is a simple form of lossless compression that is particularly useful for images with many consecutive identical pixel values, such as binary or grayscale images.

Original Size: 65536 bytes
Compressed Size: 112224 bytes
Compression Ratio: 1.71

Run Length encoding

```
% Read an image (grayscale image for simplicity)
image = imread('cameraman.tif');  % Example image (grayscale)
imshow(image);
title('Original Image');

% Flatten the 2D image into a 1D array
image_vector = image(:);

% Initialize variables
value = image_vector(1);  % First pixel value
count = 1;                % Counter for run length
encoded_values = [];      % Encoded pixel values
run_lengths = [];         % Encoded run lengths
```

Run Length encoding

```
% Run Length Encoding loop
for i = 2:length(image_vector)
    if image_vector(i) == value
        % If the current pixel matches the previous one, increase the count
        count = count + 1;
    else
        % Store the previous pixel value and its run length
        encoded_values = [encoded_values value];
        run_lengths = [run_lengths count];

        % Reset for the new pixel value
        value = image_vector(i);
        count = 1;
    end
end

% Add the last pixel value and its run length
encoded_values = [encoded_values value];
run_lengths = [run_lengths count];
```

Run Length encoding

```
% Display the encoded values and run lengths
disp('Encoded Values:');
disp(encoded_values);
disp('Run Lengths:');
disp(run_lengths);

% Compression Ratio Calculation
original_size = length(image_vector);
compressed_size = length(encoded_values) + length(run_lengths);
compression_ratio = compressed_size / original_size;

fprintf('Original Size: %d bytes\n', original_size);
fprintf('Compressed Size: %d bytes\n', compressed_size);
fprintf('Compression Ratio: %.2f\n', compression_ratio);
```

Run Length encoding

- Image Input: We load a sample grayscale image using imread.
- Flattening the Image: The 2D image is flattened into a 1D vector using image(:).
- Run Length Encoding: We loop through the image vector, counting consecutive identical pixel values (runs). When a change is detected, we store the pixel value and the run length.
- Output: The encoded values and run lengths are printed. Additionally, the compression ratio is calculated by comparing the original and compressed data sizes.

Psychovisual Redundancy

- Some visual characteristics are less important than others.
- In general observers seeks out certain characteristics – edges, textures, etc – and the mentally combine them to recognize the scene.

a b c

FIGURE 8.4
(a) Original image.
(b) Uniform quantization to 16 levels. (c) IGS quantization to 16 levels.

Psychovisual Redundancy

- If the image will only be used for visual observation, a lot of the information is usually psycho-visual redundant. It can be removed without changing the visual quality of the image. This kind of compression is usually irreversible.

- The psychovisual redundancy exist because human perception does not involve quantitative analysis of every pixel or luminance value in the image.

- It’s elimination is real visual information is possible only because the information itself is not essential for normal visual processing.

Psychovisual Redundancy

Pixel	Gray Level	Sum	IGS Code
$i-1$	N/A	0000 0000	N/A
i	0110 1100	0110 1100	0110
$i+1$	1000 1011	1001 0111	1001
$i+2$	1000 0111	1000 1110	1000
$i+3$	1111 0100	1111 0100	1111

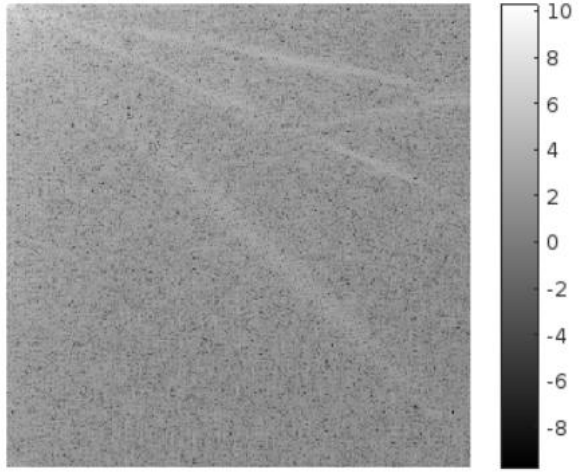
TABLE 8.2
IGS quantization procedure.

Psychovisual Redundancy

Original Image

Reconstructed Image with Psychovisual Redundancy

DCT Coefficients (Log scale)



Psychovisual Redundancy

Step 1: Read the grayscale image (for simplicity, we'll use a grayscale image)
Step2: Perform 2D Discrete Cosine Transform (DCT) to the image
Step3: Visualize the DCT coefficients

- Step 4: Zero out high-frequency components (higher psychovisual redundancy)
- % Keep only a part of the low-frequency DCT coefficients, set the rest to 0
- % Here, we'll retain only the top-left 10% of DCT coefficients.

Step 5: Apply the mask to keep only low-frequency components
Step 6: Reconstruct the image using the inverse DCT (IDCT)
Step 7: Display the reconstructed image

Fidelity Criteria

- Subjective
- Objective
  - Sum of the absolute error
  - RMS value of the error
  - Signal to Noise Ratio

Subjective scale

TABLE 8.3
Rating scale of the Television Allocations Study Organization. (Frendendall and Behrend.)

Value	Rating	Description
1	Excellent	An image of extremely high quality, as good as you could desire.
2	Fine	An image of high quality, providing enjoyable viewing. Interference is not objectionable.
3	Passable	An image of acceptable quality. Interference is not objectionable.
4	Marginal	An image of poor quality; you wish you could improve it. Interference is somewhat objectionable.
5	Inferior	A very poor image, but you could watch it. Objectionable interference is definitely present.
6	Unusable	An image so bad that you could not watch it.

Image Compression Model

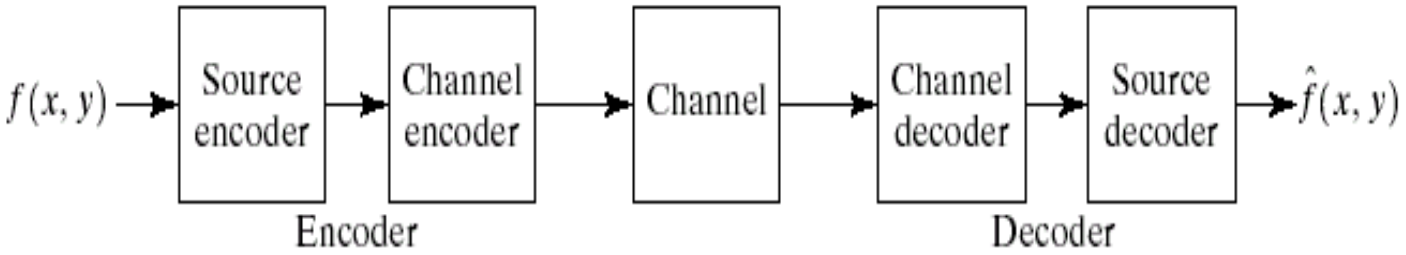
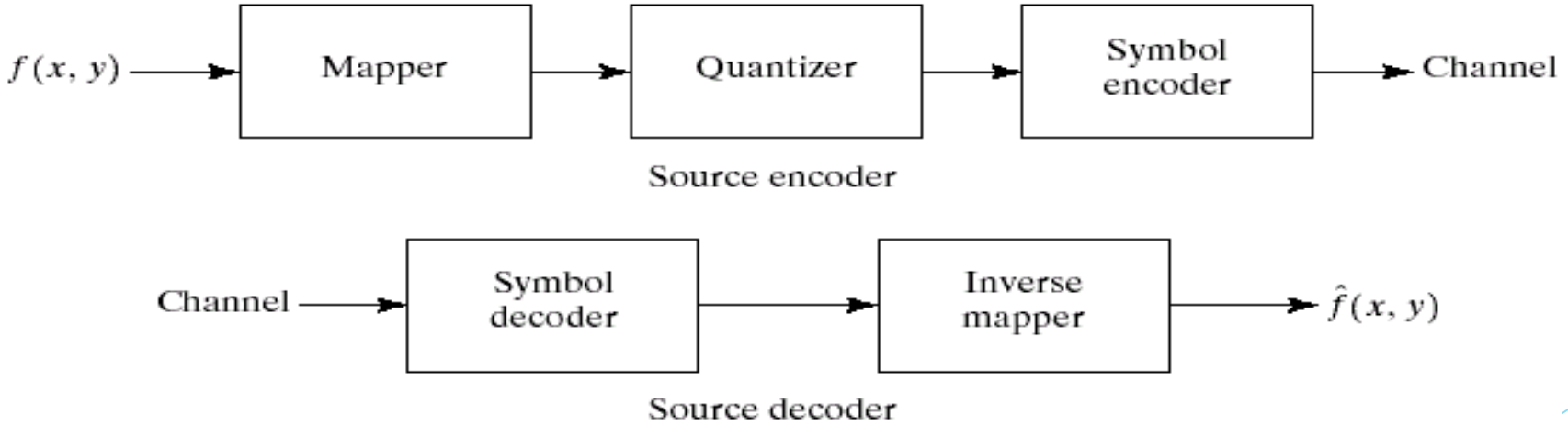


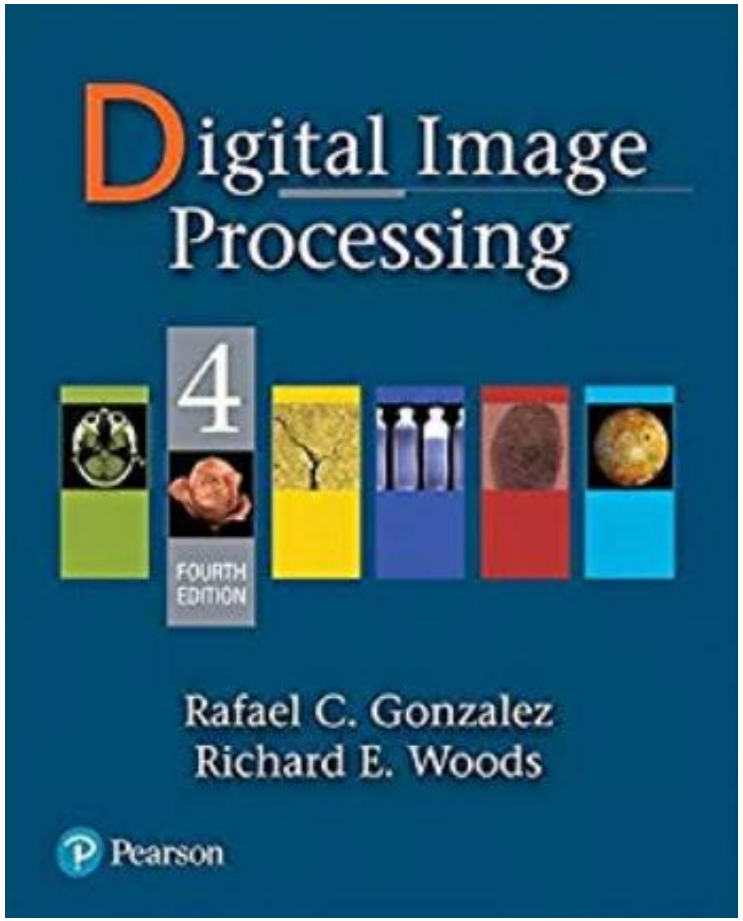
FIGURE 8.5 A general compression system model.



a
b

FIGURE 8.6 (a) Source encoder and (b) source decoder model.

Reference



Chapter 8

Question

Thank you